Practical No.01

A) Convert the Text into Tokens:

1)Code:

```
import nltk
from nltk.tokenize import word_tokenize

nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)


def tokenize_sentence(sentence):
    tokens = word_tokenize(sentence)
    return tokens

sentence = "Natural Language Processing is an interesting subject."

tokens = tokenize_sentence(sentence)

print("Original sentence:", sentence)
print("Tokens:", tokens)
print("Number of tokens:", len(tokens))
```

Output:

```
Original sentence: Natural Language Processing is an interesting subject.
Tokens: ['Natural', 'Language', 'Processing', 'is', 'an', 'interesting', 'subject', '.']
Number of tokens: 8
```

2) Taking Input from User

Code:

```
import nltk

from nltk.tokenize import word_tokenize

nltk.download('punkt')

nltk.download('punkt_tab')

text = input("Enter your text: ")

tokens = word_tokenize(text)


print("Input text:", text)

print("Tokens:", tokens)
```

```
print("Number of tokens:", len(tokens))
```

Output:

```
Enter your text:  I am Going to kathmandu
Input text: I am Going to kathmandu
Tokens: ['I', 'am', 'Going', 'to', 'kathmandu']
Number of tokens: 5
```

3) Tokenization using Function

Code:

```
import nltk

from nltk.tokenize import word_tokenize

nltk.download('punkt', quiet=True)

nltk.download('punkt_tab', quiet=True)

def tokenize_text(text):

    tokens = word_tokenize(text)

    return tokens

text = input("Enter your text: ")

result = tokenize_text(text)

print("Original text:", text)

print("Tokens:", result)

print("Number of tokens:", len(result))
```

Output:

```
Enter your text:  Lets go Mahableshwar
Original text: Lets go Mahableshwar
Tokens: ['Lets', 'go', 'Mahableshwar']
Number of tokens: 3
```

B) Find the word Frequency:

Code:

```
import nltk
```

```
from nltk.tokenize import word_tokenize

from nltk.probability import FreqDist

nltk.download('punkt', quiet=True)

nltk.download('punkt_tab', quiet=True)


def word_frequency(sentence):

    tokens = word_tokenize(sentence)

    freq = FreqDist(tokens)

    return freq


sentence = "Python is easy to learn. Python is a popular programming language."

frequency = word_frequency(sentence)


print("Original sentence:", sentence)

print("\nWord Frequency:")

for word, count in frequency.items():

    print(word, ":", count)
```

Output:

```
Original sentence: Python is easy to learn. Python is a popular programming language.

Word Frequency:
Python : 2
is : 2
easy : 1
to : 1
learn : 1
. : 2
a : 1
popular : 1
programming : 1
language : 1
```

C) Demonstrate a bigram Language Model

Code:

```
import nltk
```

```
from nltk.util import bigrams

from nltk.tokenize import word_tokenize

nltk.download('punkt', quiet=True)

nltk.download('punkt_tab', quiet=True)


sentence = "I love natural language processing"

tokens = word_tokenize(sentence)

bigram_list = list(bigrams(tokens))


print("Sentence:", sentence)

print("Bigrams:", bigram_list)
```

Output:

```
Sentence: I love natural language processing
Bigrams: [('I', 'love'), ('love', 'natural'), ('natural', 'language'), ('language', 'processing')]
```

D) Demonstrate a Trigram Language Model

Code:

```
import nltk

from nltk.tokenize import word_tokenize

from nltk.util import trigrams

nltk.download('punkt', quiet=True)

nltk.download('punkt_tab', quiet=True)


sentence = "I love natural language processing"


tokens = word_tokenize(sentence)

trigram_list = list(trigrams(tokens))


print("Sentence:", sentence)

print("Trigrams:", trigram_list)
```

Output:

```
Sentence: I love natural language processing
Trigrams: [('I', 'love', 'natural'), ('love', 'natural', 'language'), ('natural', 'language', 'processing')]
```

E) Generate Regular Expression for a Given Text

Code:

```
import re

text = "My email is abc@gmail.com and my phone number is 9876543210"

pattern = r'\w+@\w+\.\w+'

result = re.findall(pattern, text)


print("Text:", text)

print("Regular Expression:", pattern)

print("Matched Result:", result)
```

Output:

```
Text: My email is abc@gmail.com and my phone number is 9876543210
Regular Expression: \w+@\w+\.\w+
Matched Result: ['abc@gmail.com']
```

F) Text Normalization

Code:

```
import nltk

from nltk.tokenize import word_tokenize

from nltk.stem import PorterStemmer

nltk.download('punkt', quiet=True)

nltk.download('punkt_tab', quiet=True)


def normalize_text(text):
```

```
    stemmer = PorterStemmer()
    tokens = word_tokenize(text.lower())
    return [stemmer.stem(word) for word in tokens if word.isalpha()]

text = "The Cats are Playing and Running!"
result = normalize_text(text)

print("Original Text:", text)
print("Normalized Text:", result)
```

Output:

```
Original Text: The Cats are Playing and Running!
Normalized Text: ['the', 'cat', 'are', 'play', 'and', 'run']
```

Practical No.02

A) Perform Lemmatization

Code:

```
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize


nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('wordnet', quiet=True)


def lemmatize_text(text):
    lemmatizer = WordNetLemmatizer()
    words = word_tokenize(text.lower())
    return [lemmatizer.lemmatize(word) for word in words]


text = "The cats are playing with mice"
result = lemmatize_text(text)
print("Original Text:", text)
print("Lemmatized Text:", result)
```

Output:

```
Original Text: The cats are playing with mice
Lemmatized Text: ['the', 'cat', 'are', 'playing', 'with', 'mouse']
```

B) Perform Stemming:

Code:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
```

```
def stemming(text):
    stemmer = PorterStemmer()
    words = word_tokenize(text)
    return [stemmer.stem(word) for word in words]

text = "The cats are playing and running"
result = stemming(text)

print("Original Text:", text)
print("Stemmed Text:", result)
```

Output:

```
Original Text: The cats are playing and running
Stemmed Text: ['the', 'cat', 'are', 'play', 'and', 'run']
```

B) Identify Parts-of-Speech using Penn Treebank tag Set

Code:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('averaged_perceptron_tagger_eng', quiet=True)

text = "The cat is playing in the garden."
words = word_tokenize(text)
tags = pos_tag(words)
```

```
print("Word\tPOS Tag")
for word, tag in tags:
    print(word, "\t", tag)
```

Output:

```
Word    POS Tag
The      DT
cat      NN
is       VBZ
playing          VBG
in       IN
the      DT
garden   NN
.        .
```

C) Implement HMM for Pos Tagging

Code:

```
import nltk
from nltk.corpus import treebank
from nltk.tag import hmm
nltk.download('treebank', quiet=True)

data = treebank.tagged_sents()

trainer = hmm.HiddenMarkovModelTrainer()
model = trainer.train_supervised(data[:3000])

sentence = "The cat is playing"
words = sentence.split()
tags = model.tag(words)
print("Sentence:", sentence)
print("POS Tags:", tags)
```

Output:

```
Sentence: The cat is playing
POS Tags: [('The', 'DT'), ('cat', 'NNP'), ('is', 'NNP'), ('playing', 'NNP')]
```

D) Build a Chunker:

```
import nltk
from nltk import word_tokenize, pos_tag, RegexpParse
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('averaged_perceptron_tagger_eng', quiet=True)

sentence = "The quick brown fox jumps over the lazy dog."

words = word_tokenize(sentence)
tagged_words = pos_tag(words)
grammar = "NP: {<DT>?<JJ>*<NN>}"
chunker = RegexpParser(grammar)

result = chunker.parse(tagged_words)
print(result)
result.pretty_print()
```

Output:

```
(S
  (NP The/DT quick/JJ brown/NN)
  (NP fox/NN)
  jumps/VBZ
  over/IN
  (NP the/DT lazy/JJ dog/NN)
  ./.)
                              S
     _________________________|_____________________________
    |         |     |        NP               NP            NP
    |         |     |     ____|_____          |       ______|_____
jumps/VBZ over/IN ./. The/DT quick/JJ brown/NN fox/NN the/DT lazy/JJ dog/NN
```

E) Text Summarization:

Code:

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from collections import Counter
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)


def summarize(text):
    sentences = sent_tokenize(text)
    words = word_tokenize(text.lower())
    stop_words = set(stopwords.words('english'))


    freq = Counter(w for w in words
                   if w.isalpha() and w not in stop_words)
    scores = {s: sum(freq[w.lower()]
              for w in word_tokenize(s)) for s in sentences}
    return max(scores, key=scores.get)


text = """Python is a popular programming language.
It is easy to learn and use.
Python is used in data science and machine learning.
Many beginners learn Python because of its simple syntax."""
print("Summary:", summarize(text))
```

Output:

```
Summary: Many beginners learn Python because of its simple syntax.
```

Practical No.03

A) Find a Synonym of a Word using WordNet.

Code:

```
import nltk

from nltk.corpus import wordnet

nltk.download('wordnet')

word = input("Enter a word: ")


synonyms = set()


for syn in wordnet.synsets(word):

    for lemma in syn.lemmas():

        synonyms.add(lemma.name().replace('_', ' '))


synonyms.discard(word)


if synonyms:

    print("Synonyms of", word, "are:")

    print(", ".join(sorted(synonyms)))

else:

    print("No synonyms found.")
```

Output:

```
Enter a word:  happy
Synonyms of happy are:
felicitous, glad, well-chosen
```

B) Find the Antonym of a Word.

Code:

```
import nltk

from nltk.corpus import wordnet
```

```
nltk.download('wordnet')
word = input("Enter a word: ")
antonyms = set()

for syn in wordnet.synsets(word):
    for lemma in syn.lemmas():
        for antonym in lemma.antonyms():
            antonyms.add(antonym.name().replace('_', ' '))

if antonyms:
    print("Antonyms of", word, "are:")
    print(", ".join(sorted(antonyms)))
else:
    print("No antonyms found.")
```

Output:

```
Enter a word:  happy
Antonyms of happy are:
unhappy
```

C) Implement Semantic Role Labelling to Identify Named Entities.

Code:

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')

text = "Sundar Pichai visited Google headquarters in California."
```

```
tokens = nltk.word_tokenize(text)
tagged_words = nltk.pos_tag(tokens)
named_entities = nltk.ne_chunk(tagged_words)
print("Named Entities:")

for entity in named_entities:
    if hasattr(entity, 'label'):
        name = " ".join(word for word, tag in entity.leaves())
        print(name, "->", entity.label())
```

Output:

```
Named Entities:
Sundar -> PERSON
Pichai -> PERSON
Google -> GPE
California -> GPE
```

D) Resolve the Ambiguity

Code:

```
import nltk
from nltk.corpus import wordnet as wn
from nltk.wsd import lesk
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('punkt')
nltk.download('punkt_tab')

sentence = "I went to the bank to deposit money"
word = "bank"
tokens = nltk.word_tokenize(sentence)
sense = lesk(tokens, word)
```

```
print("Sentence:", sentence)

print("Ambiguous word:", word)


if sense:

    print("Resolved meaning:", sense.definition())

    print("Synonyms:", sense.lemma_names())

else:

    print("No meaning found.")
```

Output:

```
Sentence: I went to the bank to deposit money
Ambiguous word: bank
Resolved meaning: a container (usually with a slot in the top) for keeping money at home
Synonyms: ['savings_bank', 'coin_bank', 'money_box', 'bank']
```

E) Translate the Text Using First Order Logic

Code:

```
text = input("Enter a sentence: ").lower()


if text == "all humans are mortal":

    print("FOL: ∀x (Human(x) → Mortal(x))")

elif text == "some students are intelligent":

    print("FOL: ∃x (Student(x) ∧ Intelligent(x))")

elif text == "john loves mary":

    print("FOL: Loves(John, Mary)"

else:

    print("Sentence not supported")
```

Output:

```
Enter a sentence:  all humans are mortal
FOL: ∀x (Human(x) → Mortal(x))
```

Practical No.04

A) Implement RNN for sequence Labelling.

Code:

```
import torch
import torch.nn as nn
import torch.optim as optim
vocab = {
    'I': 0, 'love': 1, 'natural': 2,
    'language': 3, 'processing': 4,
    'like': 5, 'deep': 6, 'learning': 7
}

labels = ['PRON', 'VERB', 'ADJ', 'NOUN']
label_vocab = {label: i for i, label in enumerate(labels)}

X = torch.tensor([[0, 1, 2, 3, 4]], dtype=torch.long)
y = torch.tensor([[0, 1, 2, 3, 3]], dtype=torch.long)

class RNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.embedding = nn.Embedding(len(vocab), 16)
        self.rnn = nn.RNN(
            input_size=16,
            hidden_size=32,
            batch_first=True
        )
        self.fc = nn.Linear(32, len(labels))

    def forward(self, x):
```

```
        x = self.embedding(x)
        x, _ = self.rnn(x)
        return self.fc(x)

model = RNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

for epoch in range(100):
    optimizer.zero_grad()
    output = model(X)
    loss = criterion(output.reshape(-1, len(labels)), y.reshape(-1))
    loss.backward()
    optimizer.step()

print("Training completed!")
print("Final Loss:", round(loss.item(), 4))
test_words = ['I', 'like', 'deep', 'learning']
test = torch.tensor(
    [[vocab[word] for word in test_words]],
    dtype=torch.long
)

model.eval()
with torch.no_grad():
    output = model(test)
    predictions = output.argmax(dim=2)[0].tolist()
print("\nSequence Labelling:")
for word, prediction in zip(test_words, predictions):
    print(word, "->", labels[prediction])
```

Output:

```
Training completed!
Final Loss: 0.0006

Sequence Labelling:
I -> PRON
like -> VERB
deep -> ADJ
learning -> NOUN
```

B) Implement Pos Tagging Using LSTM.

Code:

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Embedding, LSTM, Dense, TimeDistributed


words = ['I', 'love', 'natural', 'language', 'processing']

word_to_id = {w: i + 1 for i, w in enumerate(words)}


tags = ['PRON', 'VERB', 'ADJ', 'NOUN', 'NOUN']

tag_to_id = {t: i for i, t in enumerate(set(tags))}


X = np.array([[word_to_id[w] for w in words]])

y = np.array([[tag_to_id[t] for t in tags]])


model = Sequential([

    Embedding(input_dim=len(word_to_id) + 1, output_dim=16),

    LSTM(32, return_sequences=True),

    TimeDistributed(Dense(len(tag_to_id), activation='softmax'))

])


model.compile(
```

```
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)


model.fit(X, y, epochs=100, verbose=0)
print("Training completed!")
predictions = model.predict(X, verbose=0).argmax(axis=-1)[0]
id_to_tag = {v: k for k, v in tag_to_id.items()}


print("\nPOS Tagging Results:")
for word, pred in zip(words, predictions):
    print(word, "->", id_to_tag[pred])
```

Output:

```
Training completed!

POS Tagging Results:
I -> VERB
love -> ADJ
natural -> NOUN
language -> NOUN
processing -> NOUN
```

C) Implement Named Entity Recognizer

Code:

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

```
text = "Sundar Pichai works at Google in California."

words = nltk.word_tokenize(text)

tagged_words = nltk.pos_tag(words)

entities = nltk.ne_chunk(tagged_words)


print("Named Entities:")

for entity in entities:

    if hasattr(entity, 'label'):

        name = " ".join(word for word, tag in entity.leaves())

        print(name, "->", entity.label())
```

Output:

```
Named Entities:
Sundar -> PERSON
Pichai -> PERSON
Google -> ORGANIZATION
California -> GPE
```

D) Word Sense Disambiguation by LSTM/GRU

Code:

```
import nltk

from nltk.wsd import lesk

from nltk.tokenize import word_tokenize


nltk.download('wordnet')

nltk.download('punkt')

nltk.download('punkt_tab')

nltk.download('omw-1.4')
```

```
def get_semantic(seq, key_word):
    tokens = word_tokenize(seq)
    sense = lesk(tokens, key_word)

    if sense:
        return sense.definition()
    else:
        return "Meaning not found"

keyword = 'book'
seq1 = 'I love reading books on coding.'
seq2 = 'The table was already booked by someone else.'
keyword1 = 'jam'

seq3 = 'My mother prepares very yummy jam.'
seq4 = 'Signal jammers are the reason for no signal.'

print("Book - Sentence 1:", get_semantic(seq1, keyword))
print("Book - Sentence 2:", get_semantic(seq2, keyword))
print("Jam - Sentence 1:", get_semantic(seq3, keyword1))
print("Jam - Sentence 2:", get_semantic(seq4, keyword1))
```

Output:

```
Book - Sentence 1: a written work or composition that has been published (printed on pag
es bound together)
Book - Sentence 2: a compilation of the known facts regarding something or someone
Jam - Sentence 1: preserve of crushed fruit
Jam - Sentence 2: deliberate radiation or reflection of electromagnetic energy for the p
urpose of disrupting enemy use of electronic devices or systems
```

Practical No.05

A) Develop Movie review system.

Code:

```
from textblob import TextBlob


def analyze_review(review):
    analysis = TextBlob(review)
    polarity = analysis.sentiment.polarity

    if polarity > 0:
        sentiment = "Positive"
    elif polarity < 0:
        sentiment = "Negative"
    else:
        sentiment = "Neutral"

    return polarity, sentiment


print("===== MOVIE REVIEW SYSTEM =====")


while True:
    review = input("\nEnter your movie review: ")
    polarity, sentiment = analyze_review(review)
    print("\nReview:", review)
    print("Polarity Score:", round(polarity, 2))
    print("Sentiment:", sentiment, "Review")

    choice = input("\nDo you want to enter another review? (yes/no): ")

    if choice.lower() != "yes":
```

```
        print("\nThank you for using Movie Review System!")
        break

reviews = [
    "This movie is amazing and wonderful",
    "The movie was boring and terrible",
    "I really enjoyed this film",
    "The acting was bad and disappointing"
]

for review in reviews:
    polarity = TextBlob(review).sentiment.polarity

    if polarity > 0:
        result = "Positive Review"
    elif polarity < 0:
        result = "Negative Review"
    else:
        result = "Neutral Review"

    print("Review:", review)
    print("Result:", result)
    print()
```

Output:

```
Enter your movie review: This movie is amazing.

Review: This movie is amazing.
Polarity Score: 0.6
Sentiment: Positive Review
```

B) Create a chatbot for HITS.

Code:

```
import re

def hits_chatbot(user_input):
    text = user_input.lower()
    text = re.sub(r'[^\w\s]', '', text)

    if any(word in text for word in ["hello", "hi", "hey"]):
        return "Hello! Welcome to HITS. How can I help you?"

    elif "admission" in text or "apply" in text:
        return "For HITS admissions, visit the official website: https://hindustanuniv.ac.in/"

    elif "course" in text or "program" in text:
        return "HITS offers programs in engineering, management, science, arts, and other fields. Check the official website for current courses."

    elif "fee" in text or "fees" in text:
        return "Course fees vary by program. Please check the official HITS website for the latest fee details."

    elif "location" in text or "address" in text:
        return "HITS is located in Padur, Kelambakkam, Chennai, Tamil Nadu, India."

    elif "contact" in text or "phone" in text:
        return "Please visit https://hindustanuniv.ac.in/ for official contact information."

    elif "thank" in text:
        return "You're welcome! Happy to help."

    elif "bye" in text or "exit" in text:
```

```
        return "Goodbye! All the best with your studies."


    else:
        return "Sorry, I don't know that yet. Please ask about HITS admissions, courses, fees, location, or contact details."


print("HITS Chatbot")
print("Type 'bye' to exit.")


while True:
    user = input("\nYou: ")
    response = hits_chatbot(user)
    print("Bot:", response)


    if "bye" in user.lower() or "exit" in user.lower():
        break
```

Output:

```
HITS Chatbot
Type 'bye' to exit.

You: BYE
Bot: Goodbye! All the best with your studies.
```